

KUBERNETES DEEP INTERNALS: THE 12-PHASE MASTER FLOW

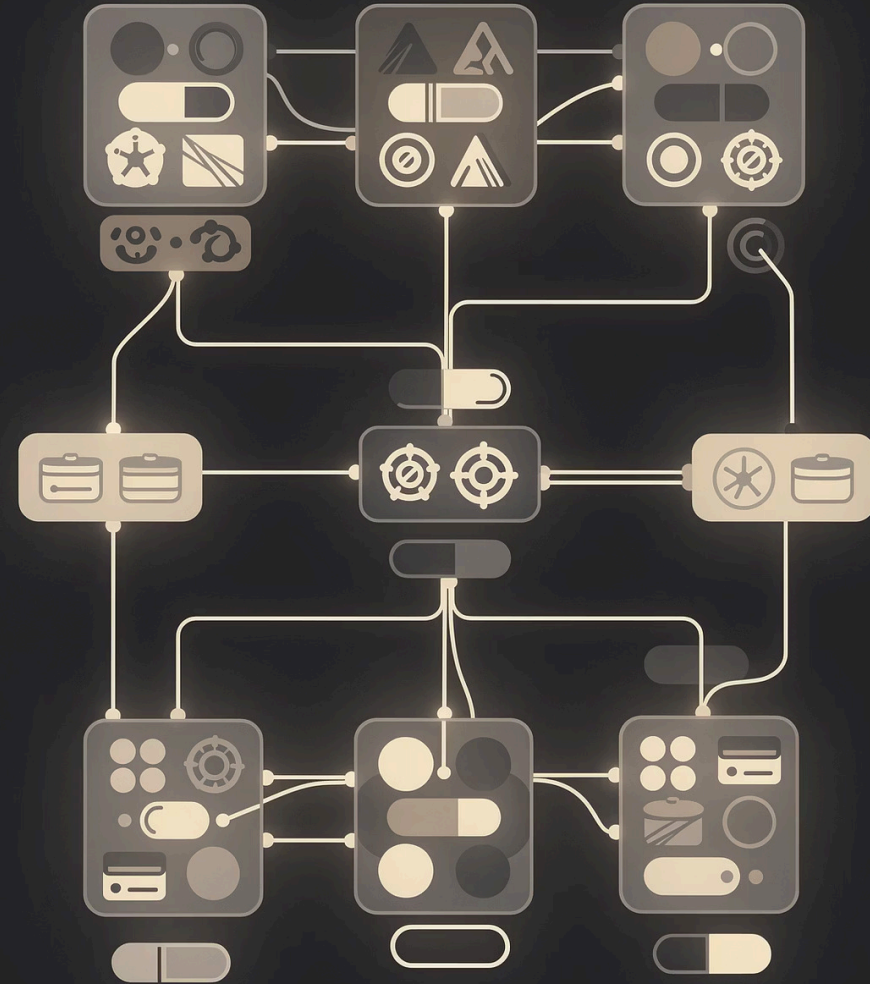
A phase-by-phase breakdown of how Kubernetes processes requests – from **kubectl apply** all the way to a routable pod – covering authentication, scheduling, runtime, networking, and service routing. Structured for engineers building CKA-level troubleshooting fluency.

CONTROL PLANE-BY RAJCOSIVA INTELLICERT TECH

NODE EXECUTION

NETWORKING

CKA LEVEL



THE 4 MASTER FLOWS – ARCHITECTURE OVERVIEW

Every Kubernetes operation can be understood through four interconnected flows. Together they form a complete end-to-end pipeline from user intent to routable workload.

FLOW 1 – CLIENT → API SERVER

How kubectl authenticates, gets authorized, passes admission control, and writes to ETCD.

FLOW 2 – CONTROL PLANE DECISION

How the Scheduler watches ETCD, filters nodes, scores them, and assigns the pod.

FLOW 3 – NODE EXECUTION

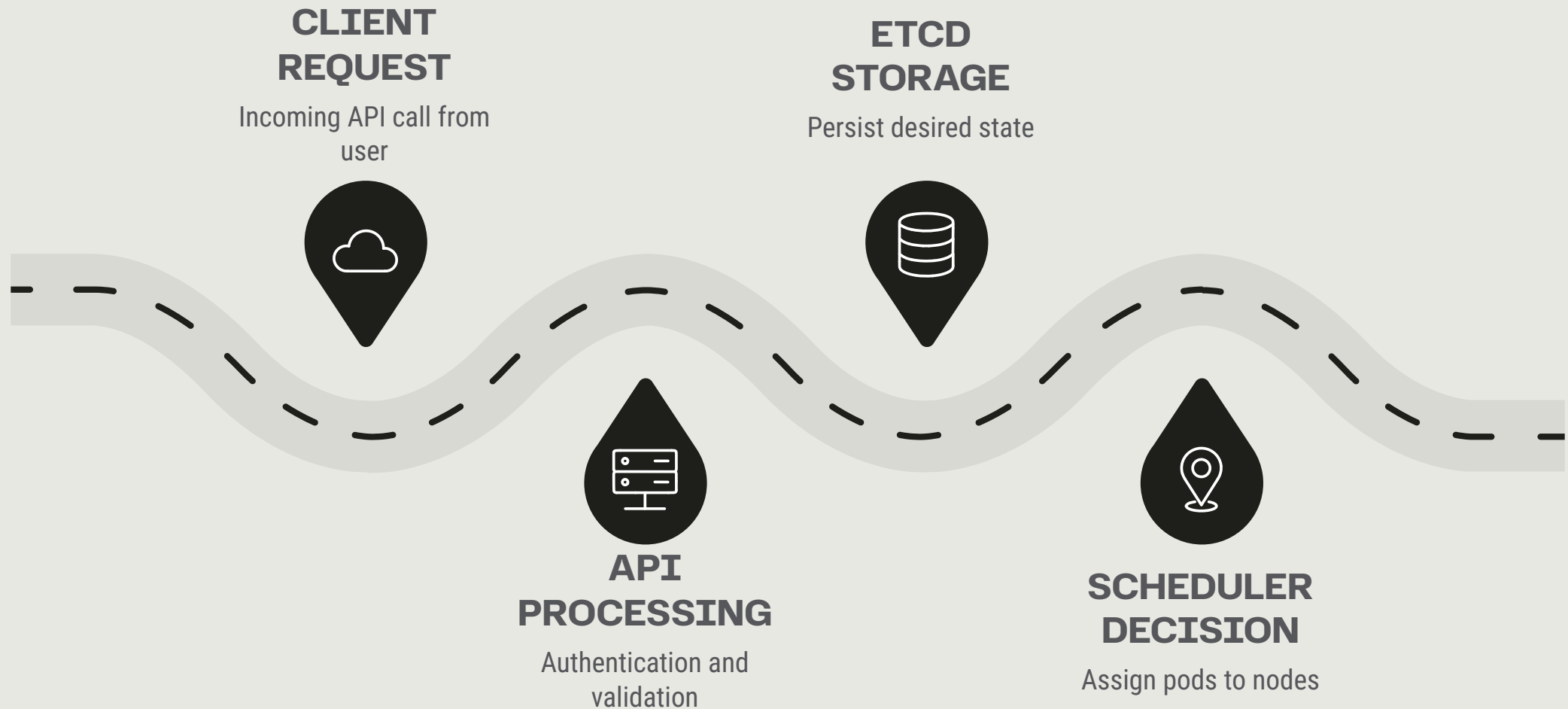
How kubelet, the container runtime, CNI, and cgroups bring a pod to life on a node.

FLOW 4 – SERVICE & NETWORKING

How Services, Endpoints, kube-proxy, and iptables/IPVS route traffic to running pods.

THE COMPLETE 12-PHASE PIPELINE

Think of Kubernetes as a **12-stage production pipeline**. Each phase has a distinct responsibility, generates specific logs, and can be independently verified. Understanding this pipeline is the foundation of expert-level Kubernetes troubleshooting.



Each phase transitions cleanly into the next — a failure at any stage propagates downstream and produces specific, diagnosable symptoms. Senior engineers learn to pinpoint exactly which phase broke.

PHASE 1: CLIENT REQUEST – KUBECTL TO API SERVER

WHAT KUBECTL DOES INTERNALLY

When you run `kubectl apply -f pod.yaml`, kubectl performs three internal steps before any network call is made:

1. Reads `~/.kube/config` – cluster endpoint, user credentials, certificate authority, and active context
2. Converts YAML → JSON (the Kubernetes API works exclusively with JSON objects)
3. Sends an HTTPS REST request: `POST /api/v1/namespaces/default/pods`

VERIFICATION COMMANDS

Inspect kubeconfig:

```
kubectl config view
```

Verify cluster connectivity:

```
kubectl cluster-info
```

Test raw API server endpoints:

```
kubectl get --raw /
```

Expected output includes: `/api /apis /version /healthz`

- ❑ All communication uses TLS. The certificate-authority field in kubeconfig is what kubectl uses to trust the API server's certificate.

PHASE 2: API SERVER – THREE SECURITY LAYERS

Every single request to the API server must pass through three sequential gates before any object is created or modified. No exceptions – not even internal cluster components bypass these layers.



1. AUTHENTICATION

Verifies *who* is making the request. Methods: Client Certificate, Bearer Token, OIDC, Webhook. `kubectl` uses client certificates by default.

Verify: `ps -ef | grep kube-apiserver` – look for `-client-ca-file` and `--service-account-key-file`



2. AUTHORIZATION (RBAC)

Determines *what* the authenticated identity can do. RBAC objects: `Role`, `ClusterRole`, `RoleBinding`, `ClusterRoleBinding`.

Verify: `kubectl auth can-i create pods` → returns `yes` or `no`

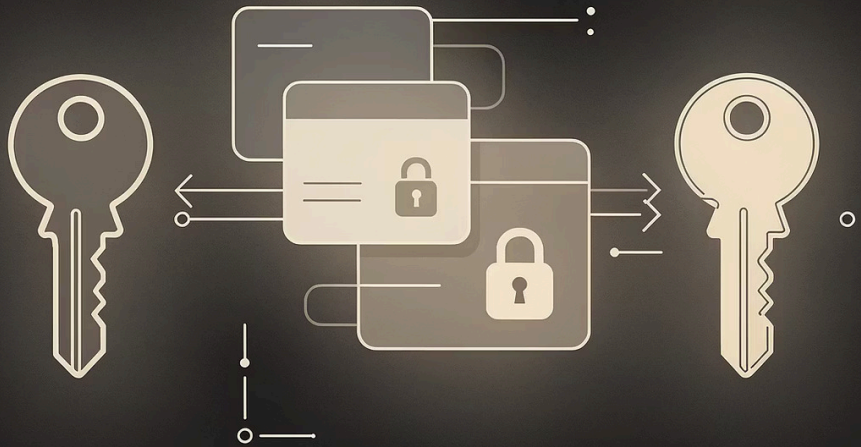


3. ADMISSION CONTROLLERS

Plugins that *mutate or validate* objects after authorization. Examples: `NamespaceLifecycle`, `LimitRanger`, `ServiceAccount`, `ResourceQuota`, `PodSecurity`.

Verify: look for `--enable-admission-plugins` in API server flags

PHASE 2 DEEP DIVE: AUTHENTICATION METHODS



CLIENT CERTIFICATE (X509)

Most common for admin access. The API server validates the certificate against `--client-ca-file`. The `CN` field becomes the username; `O` field becomes the group.

BEARER TOKEN

Used by service accounts inside the cluster. Tokens are mounted at `/var/run/secrets/kubernetes.io/serviceaccount/token` and signed with `--service-account-key-file`.

OIDC / WEBHOOK

External identity providers (e.g., Dex, Keycloak) integrate via OIDC. Webhook auth delegates verification to an external HTTP endpoint, enabling custom auth logic.

PHASE 2 DEEP DIVE: RBAC AUTHORIZATION

RBAC OBJECT MODEL

RBAC enforces least-privilege access. The four key objects form a two-level system – namespace-scoped and cluster-scoped:

- **Role** – permissions within a single namespace
- **ClusterRole** – permissions across all namespaces or cluster-wide resources
- **RoleBinding** – binds a Role to a user/group/service account in a namespace
- **ClusterRoleBinding** – binds a ClusterRole cluster-wide

 A common pattern: define a ClusterRole with read-only pod access, then bind it per-namespace using RoleBinding to limit scope.

RBAC VERIFICATION COMMANDS

```
kubectl auth can-i create pods
kubectl auth can-i delete pods -n production
kubectl get roles -A
kubectl get clusterroles
kubectl get rolebindings -A
kubectl describe clusterrole cluster-admin
```

CHECK EFFECTIVE PERMISSIONS FOR A SERVICE ACCOUNT

```
kubectl auth can-i list pods \
--as=system:serviceaccount:default:mysa
```

PHASE 3: ETCD – THE SOURCE OF TRUTH

WHAT HAPPENS IN ETCD

After passing all three security gates, the API server **persistent the pod object to ETCD**. At this moment the pod state is `Pending` – no container has started, no node has been assigned. Only the metadata exists in the key-value store.

ETCD stores all Kubernetes objects under the `/registry/` prefix, organized by resource type and namespace:

```
/registry/pods/default/nginx  
/registry/services/default/nginx-svc  
/registry/deployments/default/nginx-deploy
```

 **ETCD is the only persistent store in Kubernetes.** If ETCD data is lost without a backup, the entire cluster state is gone. Always back up ETCD in production.

VERIFICATION COMMANDS

Check pod via API server:

```
kubectl get pod nginx -o yaml
```

Verify directly in ETCD:

```
ETCDCTL_API=3 etcdctl \  
  --endpoints=https://127.0.0.1:2379 \  
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \  
  --cert=/etc/kubernetes/pki/etcd/server.crt \  
  --key=/etc/kubernetes/pki/etcd/server.key \  
  get /registry/pods --prefix --keys-only
```

Expected output:

```
/registry/pods/default/nginx
```


PHASE 4: SCHEDULER – FILTERING & SCORING

The **kube-scheduler** continuously watches the API server for pods where `nodeName == null`. When it finds one, it runs a two-phase algorithm to select the best node.

PHASE A: FILTERING

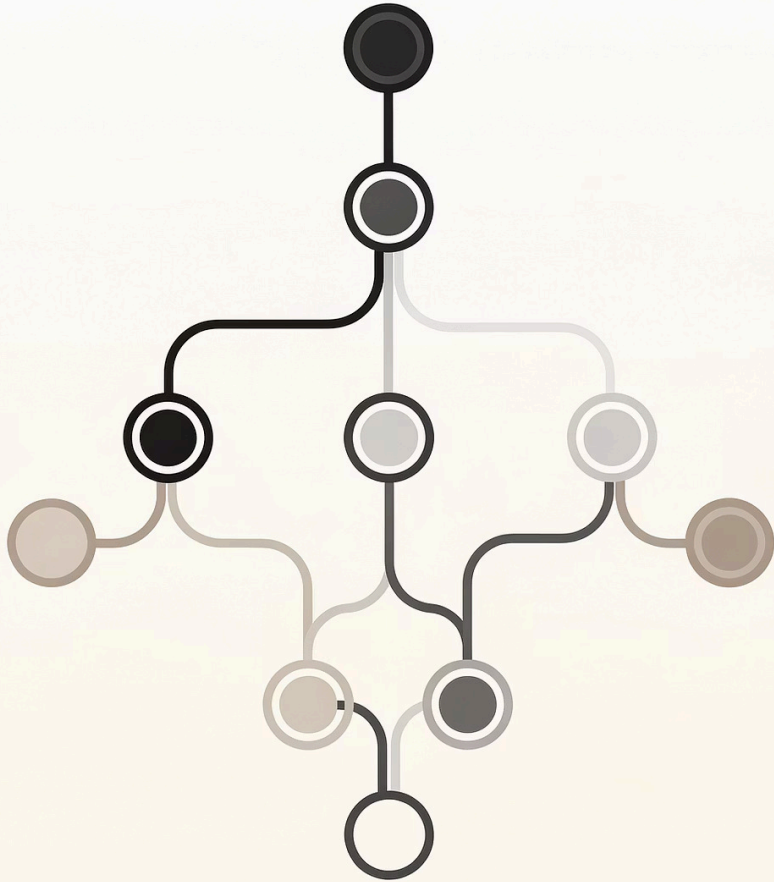
Eliminates nodes that *cannot* run the pod. Checks include: sufficient CPU and memory, NodeSelector match, Taint/Toleration compatibility, node readiness, volume binding availability, and pod affinity/anti-affinity rules.

Example: node1 ✓ | node2 ✗ (memory full) | node3 ✓

PHASE B: SCORING

Ranks filtered nodes using scoring functions: LeastRequested (prefers least-loaded), BalancedAllocation (balances CPU/memory ratio), ImageLocality (prefers nodes that already have the image).

Example: node1 = 65 pts | node3 = 92 pts → **node3 wins**



PHASE 4 DEEP DIVE: SCHEDULING CONSTRAINTS



TAINTS & TOLERATIONS

Taints repel pods from nodes. A pod must declare a matching `toleration` to be scheduled onto a tainted node. Common use: dedicate nodes to specific workloads like GPU jobs or system daemons.



TOPOLOGY SPREAD

`topologySpreadConstraints` distributes pods evenly across failure domains (zones, nodes). Critical for high-availability deployments where skew between zones must be minimized.

VERIFICATION



NODE AFFINITY

Expresses preference or requirement for node labels. `requiredDuringScheduling` is a hard constraint; `preferredDuringScheduling` influences scoring. More expressive than `NodeSelector`.



RESOURCE REQUESTS

Scheduler uses `requests` (not `limits`) to determine if a node has capacity. Pods without resource requests may land on overloaded nodes – always define them in production.

```
kubectl get pod nginx -o wide
journalctl -u kube-scheduler | grep nginx
# Expected: Successfully assigned default/nginx to node3
```

PHASE 5: KUBELET – THE NODE AGENT

WHAT KUBELET DOES

Once the scheduler writes `nodeName: node3` to ETCD, the **kubelet** running on `node3` detects the assignment via its API server watch loop. Kubelet is the primary node agent – it owns the full pod lifecycle on that node.

Kubelet's responsibilities include:

- Receiving pod specs from the API server (or static pod files)
- Instructing the container runtime to pull images and start containers
- Calling CNI plugins to configure networking
- Mounting volumes and secrets/configmaps
- Running liveness and readiness probes
- Reporting pod status back to the API server

KUBELET VERIFICATION

Check kubelet service status:

```
systemctl status kubelet
```

Tail kubelet logs in real time:

```
journalctl -u kubelet -f
```

Key log entries to look for:

```
SyncLoop ADD pod default/nginx  
Pulling image "nginx:latest"  
Successfully pulled image  
Created container nginx  
Started container nginx
```

Kubelet config location:

```
/var/lib/kubelet/config.yaml  
/etc/kubernetes/kubelet.conf
```

- ❏ Each node runs exactly one kubelet process. It is the only Kubernetes component that runs directly on the node OS – not inside a pod.

PHASE 6: POD SANDBOX & CONTAINER RUNTIME

Before any application container starts, kubelet creates a **Pod Sandbox** using the `pause` container. This establishes the shared kernel namespaces that all containers in the pod will inherit.

THE PAUSE CONTAINER

The `pause` container (also called the infra container) holds the **Network** and **IPC** namespaces. It runs a minimal process that does nothing except keep the namespace alive. All app containers join its namespaces – that's why they share an IP and can communicate via `localhost`.

Pod Structure:

- └── `pause` container ← holds namespaces
- └── `nginx` container ← joins namespaces
- └── `sidecar` container ← joins namespaces

CRI – CONTAINER RUNTIME INTERFACE

Kubelet communicates with the runtime via the **CRI gRPC API**. Supported runtimes: `containerd` (default in kubeadm clusters) and `CRI-O`. Docker is no longer supported natively since Kubernetes 1.24.

CRI calls issued by kubelet in order: `RunPodSandbox` → `PullImage` → `CreateContainer` → `StartContainer`

VERIFICATION

```
crictl ps # list all running containers including pause
crictl images # verify image was pulled
ctr containers list # containerd native CLI
journalctl -u containerd -f # container runtime logs
```

PHASE 7: CNI NETWORKING – POD GETS AN IP

WHAT CNI DOES

Before the application container starts, kubelet calls the **CNI (Container Network Interface)** plugin to set up networking. CNI plugins are binary executables stored in `/opt/cni/bin/`.

CNI executes these steps in order:

1. Create a new **network namespace** for the pod
2. Create a **veth pair** – one end (`eth0`) inside the pod namespace, one end (`vethXXXX`) on the node
3. Assign a **pod IP** (e.g., `10.244.1.5`) from the pod CIDR
4. Configure **routes** so the pod can reach other pods and services
5. Program **iptables rules** or eBPF hooks (depending on CNI plugin)

COMMON CNI PLUGINS

- **Calico** – BGP-based routing, supports NetworkPolicy, widely used in production
- **Flannel** – simple VXLAN overlay, easy to set up
- **Cilium** – eBPF-based, high performance, L7 policy support
- **Weave** – mesh networking with encryption

VERIFICATION COMMANDS

```
kubectl get pod nginx -o wide # pod IP
ip link # see vethXXXX + cni0
ls /etc/cni/net.d/ # CNI config files
# e.g.: 10-calico.conflist
ip route # routing table on node
```

PHASE 7 DEEP DIVE: LINUX NAMESPACES & CGROUPS

NETWORK NAMESPACE

Each pod gets its own netns – an isolated network stack with its own interfaces, routes, and iptables rules. The veth pair bridges the pod namespace to the node network.

```
lsns          # list all namespaces
# 4026532447 net ← pod namespace
nsenter -t -n ip addr
```

PID & IPC NAMESPACE

Containers in a pod optionally share PID and IPC namespaces. Sharing PID namespace allows containers to see each other's processes, which is useful for sidecar debugging scenarios.

```
crictl inspect \
| grep -i namespace
```

CGROUPS – RESOURCE CONTROL

Linux **cgroups** (control groups) enforce CPU and memory limits at the kernel level. Kubelet creates a cgroup hierarchy per pod and per container. OOMKill events appear in kernel logs when a container exceeds its memory limit.

```
ls /sys/fs/cgroup/memory/kubepods/
# guaranteed/ burstable/ besteffort/
dmesg | grep -i oom
```

VOLUME MOUNTS

Kubelet mounts volumes before starting containers. Pod volume data lives at `/var/lib/kubelet/pods/<uid>/volumes/`. ConfigMaps and Secrets are projected as tmpfs mounts.

```
ls /var/lib/kubelet/pods/
mount | grep /var/lib/kubelet
```

PHASE 8: POD RUNNING STATE

WHAT "RUNNING" ACTUALLY MEANS

Once the container process starts, kubelet performs an internal reconciliation loop (the **SyncLoop**) that continuously compares *desired state* from the API server with *actual state* on the node. When container processes are up, kubelet updates the pod's `status.phase` to `Running` and posts container-level conditions to the API server.

Key pod status conditions:

- `PodScheduled` – node was assigned
- `Initialized` – init containers completed
- `ContainersReady` – all containers passed readiness
- `Ready` – pod is in service

VERIFICATION – POD EVENTS TIMELINE

```
kubectl describe pod nginx
```

Events section shows the full lifecycle:

Events:

```
Normal Scheduled   node3
Normal Pulling     image "nginx"
Normal Pulled      image pulled
Normal Created     container nginx
Normal Started     container nginx
```

CHECK POD CONDITIONS DIRECTLY:

```
kubectl get pod nginx -o yaml \
| grep -A 20 conditions
```

- ❏ If a pod is stuck in Pending, check `kubectl describe pod` – the Events section will tell you exactly which phase failed (e.g., insufficient memory, image pull failure).

PHASE 9 & 10: SERVICE CREATION & ENDPOINTS

A pod running with an IP is not yet reachable from other workloads or users in a stable, discoverable way. The **Service** resource provides a stable virtual IP (ClusterIP) that persists even as pods are replaced.

KUBECTL EXPOSE

User creates a Service: `kubectl expose pod nginx --port=80`. A Service object is written to ETCD with a virtual **ClusterIP** (e.g., `10.96.0.20`) assigned by the API server from the service CIDR.

1

ENDPOINTS LICES

In modern Kubernetes (1.19+), `EndpointSlice` objects replace large `Endpoints` objects. Slices support up to 100 endpoints each, improving scalability for large deployments.

2

3

4

ENDPOINT CONTROLLER

The **Endpoints Controller** inside `kube-controller-manager` watches for new Services and matching pods. It creates an `Endpoints` object mapping the Service to live pod IPs.

VERIFICATION

`kubectl get svc` → see ClusterIP. `kubectl get endpoints` → see pod IP mapping: `nginx-service` → `10.244.1.5:80`

PHASE 11: KUBE-PROXY – IPTABLES/IPVS RULE PROGRAMMING

WHAT KUBE-PROXY DOES

kube-proxy runs as a DaemonSet on every node. It watches the API server for Service and Endpoints/EndpointSlice changes. When a new Service is created, kube-proxy programs the kernel's packet-filtering layer to redirect traffic from the ClusterIP to one of the backing pod IPs.

Two modes:

- **iptables mode** (default) – programs DNAT rules in `nat` table. Stateless, uses random or RR selection.
- **IPVS mode** – uses Linux Virtual Server for higher performance and richer load-balancing algorithms (rr, lc, sh, etc.).

📌 Cilium CNI can replace kube-proxy entirely using eBPF, eliminating iptables overhead for very large clusters.

IPTABLES CHAIN STRUCTURE

```
iptables -t nat -L -n --line-numbers
```

Key chains created by kube-proxy:

KUBE-SERVICES	← entry point for service IPs
KUBE-NODEPORTS	← NodePort rules
KUBE-SVC-XXXX	← per-service chain
KUBE-SEP-XXXX	← per-endpoint (DNAT)

Trace a specific service:

```
iptables -t nat -L KUBE-SERVICES -n \
| grep 10.96.0.20
```

VERIFY KUBE-PROXY SERVICE:

```
systemctl status kube-proxy
journalctl -u kube-proxy | grep nginx
```

PHASE 12: PACKET ROUTING – REQUEST TO POD

This is the deepest level of Kubernetes networking – the full kernel path a packet takes from a client to an application container. Understanding this is essential for CKA and CKS-level troubleshooting.

VERIFY END-TO-END FROM ANOTHER POD

```
kubectl exec -it busybox -- curl 10.96.0.20  
kubectl exec -it busybox -- curl http://nginx-service.default.svc.cluster.local
```

- ❏ DNS resolution in Kubernetes is handled by CoreDNS. Service DNS follows the pattern: <service>.<namespace>.svc.cluster.local. Pod DNS: <pod-ip-dashes>.<namespace>.pod.cluster.local.

SERVICE TYPES: NODEPORT, CLUSTERIP & LOADBALANCER

CLUSTERIP (DEFAULT)

Exposes the service on an internal virtual IP reachable only within the cluster. This is the foundation for all other service types. No external access.

```
kubectl get svc nginx
# TYPE: ClusterIP
# CLUSTER-IP: 10.96.0.20
```

NODEPORT

Exposes the service on a static port (30000–32767) on every node's IP. Traffic to NodeIP:NodePort is forwarded to the ClusterIP and then to pods. Useful for development; not recommended for production external access.

```
kubectl get svc nginx
# EXTERNAL: :30007
```

LOADBALANCER

Provisions an external load balancer from the cloud provider (AWS ELB, GCP LB, Azure LB). It builds on top of NodePort. On bare metal, use **MetalLB** to achieve the same behavior.

```
kubectl get svc nginx
# EXTERNAL-IP: 34.120.10.5
```

EXTERNALNAME

Maps a service to a DNS name rather than a pod selector. Used to alias external services like a managed database into the cluster namespace. No proxying occurs – it's pure DNS CNAME.

```
spec:
  type: ExternalName
  externalName: db.corp.com
```

THE COMPLETE 4-FLOW ARCHITECTURE – UNIFIED VIEW

Every pod creation and every user request traverses this exact path. Memorizing this chain is the single most effective way to build Kubernetes troubleshooting intuition.

WHY THE 4-FLOW MODEL OUTPERFORMS TRADITIONAL TEACHING

TRADITIONAL APPROACH

Most Kubernetes courses teach components in isolation: "Here is the scheduler. Here is kubelet. Here is a Service." This creates engineers who can define each component but cannot answer: *"Why is my pod stuck in Pending?"* or *"Why can't my service route traffic?"*

✗ ISOLATED COMPONENT KNOWLEDGE

Can define components but cannot trace failures across phase boundaries.

✗ MEMORIZED COMMANDS

Knows commands but doesn't know *why* they reveal specific information at each phase.

THE 4-FLOW MODEL

By following the request pipeline end to end, engineers build a **mental model that maps symptoms to phases**. When something breaks, you know exactly which log to check and which phase to inspect.

✓ PHASE-AWARE DEBUGGING

A pod stuck in Pending = Phase 4 (Scheduler) or Phase 3 (ETCD) issue – not a runtime problem.

✓ LOG-TO-PHASE MAPPING

Each phase generates specific logs. Engineers know exactly where to look without guessing.

COMPLETE DEBUG FLOW – PRODUCTION TROUBLESHOOTING ORDER

Senior Kubernetes engineers always follow a **phase-ordered debug sequence**. Starting from Phase 1 and moving forward prevents wasting time checking kube-proxy when the problem is actually a scheduler constraint.

Θ1

API SERVER – KUBECTL GET EVENTS

First stop: cluster-wide events show rejected requests, quota violations, and admission controller rejections.

```
kubectl get events --sort-by=.lastTimestamp -A
```

Θ2

SCHEDULER – KUBECTL DESCRIBE POD

Unschedulable pods display the reason in the Events section. Look for insufficient CPU/memory, taint mismatches, or affinity failures.

```
kubectl describe pod nginx | grep -A 10 Events
```

Θ3

KUBELET – JOURNALCTL

If pod is scheduled but not starting, kubelet logs show image pull errors, volume mount failures, or CNI errors.

```
journalctl -u kubelet --since "5 min ago"
```

Θ4

RUNTIME – CRICTL

Container runtime reveals OCI spec errors, image filesystem issues, or seccomp/AppArmor denials.

```
crictl ps -a # all containers, including exited  
crictl inspect
```

Θ5

NETWORK – IP LINK + IPTABLES

Missing veth pairs indicate CNI failure. Missing KUBE-SERVICES entries indicate kube-proxy is not running or has not synced.

```
ip link | grep veth  
iptables -t nat -L KUBE-SERVICES -n
```

Θ6

SERVICE – KUBECTL GET ENDPOINTS

An empty Endpoints object means no pods match the Service selector – check labels on pods vs. selector on Service.

```
kubectl get endpoints nginx-service
```

KEY VERIFICATION COMMANDS – QUICK REFERENCE

Phase	Command	What You're Verifying
Phase 1	<code>kubectl config view</code>	kubeconfig, cluster endpoint, credentials
Phase 2	<code>ps -ef grep kube-apiserver</code>	Auth flags: <code>--client-ca-file</code> , <code>--enable-admission-plugins</code>
Phase 2	<code>kubectl auth can-i create pods</code>	RBAC permission check for current identity
Phase 3	<code>etcdctl get /registry/pods --prefix --keys-only</code>	Pod object persisted to ETCD
Phase 4	<code>kubectl get pod -o wide</code>	nodeName assigned by scheduler
Phase 4	<code>journalctl -u kube-scheduler</code>	Scheduling decisions and failures
Phase 5	<code>journalctl -u kubelet</code>	SyncLoop events, volume mount, CNI calls
Phase 6	<code>crictl ps</code> / <code>crictl images</code>	Containers running, images pulled
Phase 7	<code>ip link</code> / <code>ip route</code>	veth pairs created, routes configured
Phase 8	<code>kubectl describe pod nginx</code>	Full event timeline, container state
Phase 9	<code>kubectl get svc</code>	ClusterIP assigned, service type, ports
Phase 10	<code>kubectl get endpoints</code>	Pod IPs mapped to service
Phase 11	<code>iptables -t nat -L -n</code>	KUBE-SERVICES and KUBE-SEP chains
Phase 12	<code>kubectl exec busybox -- curl 10.96.0.20</code>	End-to-end packet routing validated

COREDNS – THE MISSING LINK IN SERVICE DISCOVERY

HOW DNS WORKS INSIDE THE CLUSTER

When a pod runs `curl http://nginx-service`, the kernel resolves it via the pod's `/etc/resolv.conf`, which points to the **CoreDNS** ClusterIP. CoreDNS responds with the Service's ClusterIP, not the pod IP directly.

DNS search domains in pod `resolv.conf`:

```
search default.svc.cluster.local \
      svc.cluster.local \
      cluster.local
nameserver 10.96.0.10
```

This is why `curl nginx-service` works from within the same namespace – the search domain appends `.default.svc.cluster.local` automatically.

DNS RECORD PATTERNS

- **Service A record:** `nginx-svc.default.svc.cluster.local` → `10.96.0.20`
- **Pod A record:** `10-244-1-5.default.pod.cluster.local` → `10.244.1.5`
- **Headless service:** returns pod IPs directly instead of ClusterIP

VERIFY COREDNS

```
kubectl get pods -n kube-system \
-l k8s-app=kube-dns
kubectl logs -n kube-system \
-l k8s-app=kube-dns
```

```
# Test DNS from inside a pod:
kubectl exec -it busybox -- \
nslookup nginx-service
```


STATIC PODS & CONTROL PLANE BOOTSTRAP

Understanding where control plane components come from is critical for troubleshooting master node failures. In kubeadm clusters, the control plane runs as **static pods** – not managed by the API server itself, but by kubelet directly.



STATIC POD MANIFESTS

Control plane component YAML files live at `/etc/kubernetes/manifests/`. Kubelet watches this directory. Any YAML file here is launched as a pod, even if the API server is down. Editing a file here immediately restarts the component.

```
ls /etc/kubernetes/manifests/  
# etcd.yaml  
# kube-apiserver.yaml  
# kube-controller-manager.yaml  
# kube-scheduler.yaml
```



PKI CERTIFICATES

All control plane TLS certificates are stored at `/etc/kubernetes/pki/`. The API server requires several certificate pairs for different communication channels: etcd client, kubelet client, front-proxy, and the main server cert.

```
ls /etc/kubernetes/pki/  
# ca.crt ca.key  
# apiserver.crt apiserver.key  
# etcd/ front-proxy-ca.crt
```



CONTROLLER MANAGER LOOPS

`kube-controller-manager` runs dozens of reconciliation loops: ReplicaSet, Deployment, Endpoints, Job, ServiceAccount, and more. Each loop watches ETCD for its resource type and drives actual state toward desired state.

```
kubectl get pods -n kube-system  
# kube-controller-manager-master  
journalctl -u kube-controller-manager
```

POD LIFECYCLE STATE MACHINE – FROM PENDING TO TERMINATED

CRASHLOOPBACKOFF EXPLAINED

Not an official pod phase – it's a container state. Kubernetes restarts crashed containers with exponential backoff: 10s, 20s, 40s, 80s, up to 5 minutes. After enough restarts, the pod status shows `CrashLoopBackOff`.

```
kubectl describe pod nginx | grep -i back-off  
kubectl logs nginx --previous # previous container logs
```

INIT CONTAINERS

Init containers run sequentially before app containers start. They must exit with code 0 before the next init container or app container starts. Useful for pre-flight checks: waiting for a database, downloading config, or running migrations.

```
kubectl get pod nginx -o yaml \  
| grep -A 5 initContainerStatuses
```

NETWORKPOLICY – CONTROLLING POD-TO-POD TRAFFIC

WHAT NETWORKPOLICY DOES

By default, all pods can communicate with all other pods in a Kubernetes cluster. **NetworkPolicy** resources define L3/L4 ingress and egress rules that CNI plugins enforce at the kernel level (using iptables or eBPF).

A pod becomes *isolated* the moment any NetworkPolicy selects it – all traffic not explicitly allowed is then denied.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
spec:
  podSelector:
    matchLabels:
      app: nginx
  ingress:
    - from:
        - podSelector:
            matchLabels:
              role: frontend
      ports:
        - port: 80
```

DEBUGGING NETWORKPOLICY

NetworkPolicy enforcement is done entirely by the CNI plugin. If your CNI (e.g., Flannel) does not support NetworkPolicy, the policies exist in ETCD but have **no effect**. Use Calico, Cilium, or Weave for enforcement.

VERIFY POLICIES:

```
kubectl get networkpolicies -A
kubectl describe networkpolicy deny-all
```

TEST CONNECTIVITY:

```
kubectl exec -it pod-a -- \
  curl http://pod-b-ip:80
# If blocked → no response or "Connection refused"
```

📄 Cilium provides **cilium monitor** to trace which policy is dropping packets – invaluable for production debugging.

PROBES – HOW KUBERNETES KNOWS A CONTAINER IS HEALTHY



LIVENESS PROBE

Answers: *"Is this container still alive?"* If the liveness probe fails consecutively (default: 3 times), kubelet kills and restarts the container. Use to detect deadlocks or infinite loops where the process is running but not functioning.

```
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5
```



READINESS PROBE

Answers: *"Is this container ready to receive traffic?"* Failing readiness removes the pod IP from the Endpoints object, preventing kube-proxy from routing traffic to it. The pod is NOT restarted – it just stops receiving traffic until healthy again.

```
readinessProbe:
  tcpSocket:
    port: 3306
  initialDelaySeconds: 5
  periodSeconds: 10
```



STARTUP PROBE

Answers: *"Has the application started yet?"* Disables liveness and readiness probes until the startup probe succeeds. Critical for slow-starting apps (e.g., JVM applications) that would be killed by a liveness probe before initialization completes.

```
startupProbe:
  httpGet:
    path: /startup
    port: 8080
  failureThreshold: 30
  periodSeconds: 10
```

RESOURCE MANAGEMENT – REQUESTS, LIMITS & QOS CLASSES

REQUESTS VS. LIMITS

Requests – the amount of CPU/memory the scheduler uses to find a fitting node. The node must have at least this much available. Guaranteed to the container by the kernel (via cgroups).

Limits – the maximum. CPU limits are throttled (not killed); memory limits cause an OOMKill if exceeded.

```
resources:
  requests:
    cpu: "250m"
    memory: "128Mi"
  limits:
    cpu: "500m"
    memory: "256Mi"
```

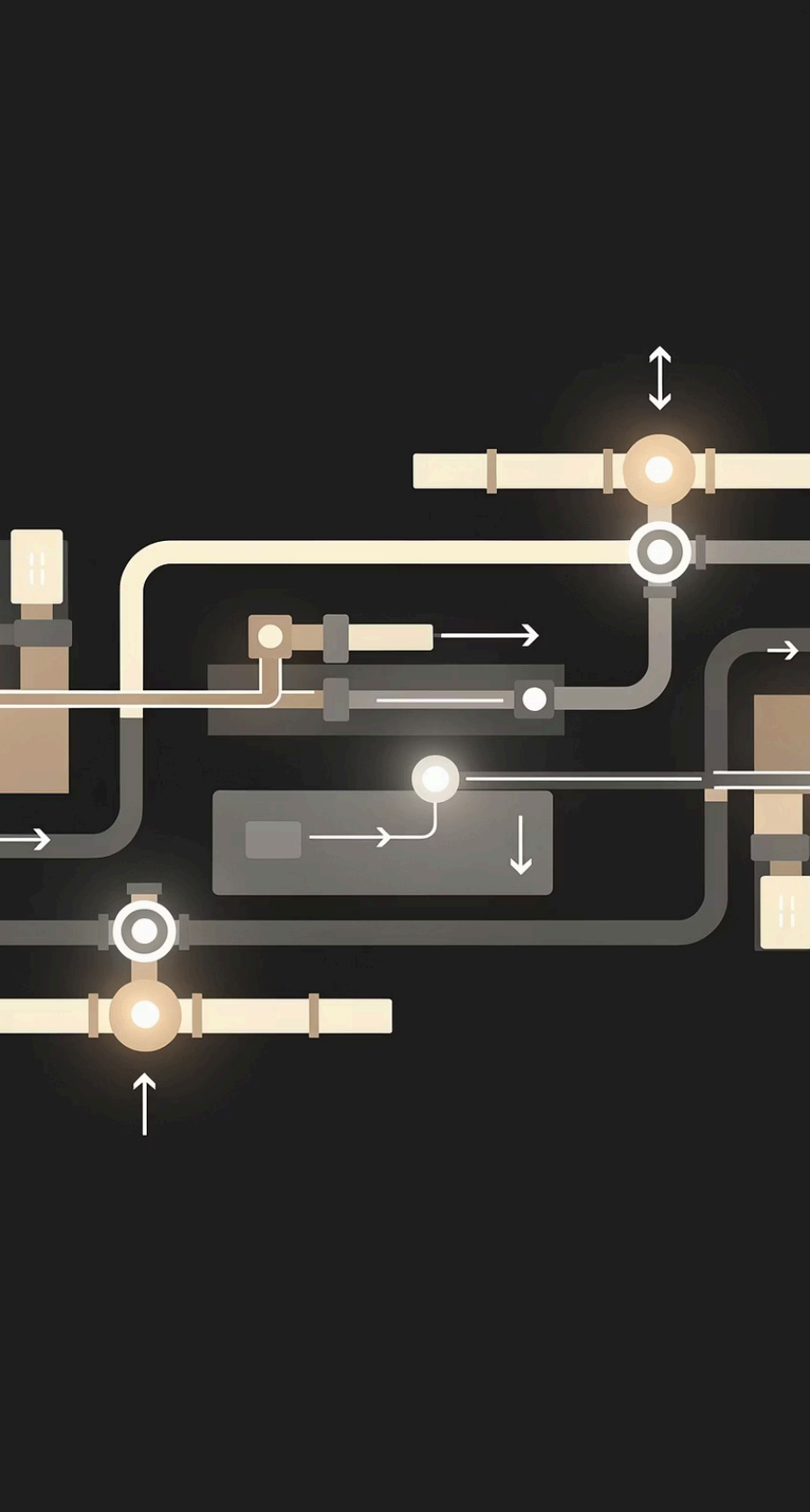
QOS CLASSES

Kubernetes assigns one of three QoS classes to each pod, which determines eviction priority under node memory pressure:

- **Guaranteed** – requests == limits for all containers. Last to be evicted.
- **Burstable** – requests set but limits are higher or unset. Medium priority.
- **BestEffort** – no requests or limits set. First to be evicted.

```
kubectl get pod nginx -o yaml \
  | grep qosClass
# qosClass: Guaranteed
```

```
dmesg | grep -i oom
# detect OOMKill events on node
```



FULL MENTAL MODEL – THE KUBERNETES REQUEST PIPELINE

Think of Kubernetes as a **12-stage production pipeline** where each stage is independently observable, debuggable, and has a specific responsibility. Mastery comes not from memorizing YAML, but from understanding what happens at each boundary.

12

PIPELINE PHASES

From kubectl apply to routed user request

3

SECURITY LAYERS

Auth, RBAC, and Admission Controllers on every request

4

MASTER FLOWS

Client, Control Plane, Node Execution, Networking

10

DEBUG STEPS

Ordered troubleshooting from API server to iptables

Every time something breaks in Kubernetes, it breaks at exactly one of these 12 phases. Knowing the pipeline means knowing where to look – and that is what separates an operator from an engineer.